Silicon Shoppers: An Adaptive Storefront the Shopper Can Operate

Prabhu Pugalenthi
University of Southern California
Los Angeles, CA, USA
pugalent@usc.edu

Ansel Zeng
University of Southern California
Los Angeles, CA, USA
anselzen@usc.edu

Sumin Jo
University of Southern California
Los Angeles, CA, USA
suminjo@usc.edu

ABSTRACT

E-commerce personalization stops at the recommendation rail: the store itself looks identical for every shopper, and when the underlying model is wrong the shopper has no recourse. We present *Silicon Shoppers*, a deployed web storefront in which the entire layout (navigation order, hero advertisements, homepage rails, listing sort, filters, badges, and checkout variant) is a typed, versioned document that three different writers can mutate: a purchase-history model, a live browsing model, and the shopper, speaking to an LLM concierge. The concierge is not decorative: it emits structured actions that are validated, executed, and audited against real store state, so a reply that claims an action the model did not emit is either recovered into a real state change or honestly corrected. A staged animation engine plays every layout change component by component, making the rearrangement itself inspectable. Attendees can chat with the concierge, rearrange the store in natural language, complete a purchase end to end, and watch every model decision, layout step, and agent action narrated live on a companion terminal.

CCS CONCEPTS

• **Information systems** → **Users and interactive retrieval**; *Recommender systems*; • **Human-centered computing** → *Interactive systems and tools.*

KEYWORDS

personalization, LLM agents, conversational commerce, adaptive interfaces, tool-use faithfulness

1 INTRODUCTION

Online personalization has converged on a single shape: a fixed storefront with a recommendation shelf stapled on. The navigation, the hero banner, the order of homepage sections, the sort order of a listing, the structure of the store, is identical for a first-time browser and a decade-long customer. Meanwhile, a new generation of LLM agents is being deployed into consumer settings, where they do more than answer questions: they add items to carts, apply filters, and initiate payments. Two problems meet here. First, shoppers cannot talk back to the store; the model's decisions are opaque and unappealable. Second, LLM agents that act are unreliable narrators of their own behavior: they routinely report actions they never executed [1, 2].

Silicon Shoppers is a personalization engine, demonstrated inside a fashion storefront we built (SoCal Atelier), around a simple idea: the store layout itself is data, and an agent can act on it. Three writers mutate one typed, versioned layout contract: (i) a model trained on purchase history merchandises the store before first paint; (ii) a live browsing model refreshes it every four minutes from in-session events; and (iii) the shopper, through a concierge chat, rearranges the store in natural language ("make this more men-friendly, navy forward") or delegates whole tasks to it ("buy the wool trouser in black, size M"). Every change plays as a staged animation, and every model input, layout output, and event persists to Snowflake.

System architecture: layout contract, three writers, honesty layer, data plane.

Figure 1: Silicon Shoppers architecture. All layout mutation flows through one versioned contract; all mutation events are validated, staged, and logged.

The demo's technical contribution is the agent's *honesty layer*: a pipeline in which the agent's actions are validated against the real catalog before it writes its reply, recovered from claim prose when absent, and audited against actual state transitions before any claim reaches the shopper. Attendees interact with a real, working deployment, not a screencast.

2 SYSTEM OVERVIEW

Figure 1 shows the architecture. The system has three surfaces (the storefront, a simulator shell that hosts and instruments it, and a data plane that persists everything) connected by a single typed layout contract.

2.1 The layout contract

The storefront renders entirely from one versioned JSON document (`atelier.store_layout.v1`): navigation order and highlight, hero and secondary ad placements, homepage blocks (rails, grids, offer strips), listing sort and filter state, badge emphasis, and checkout variant. Every mutation is schema-validated, so no writer, model or human, can produce a malformed store. Because layout is data rather than code, it can be diffed, replayed, audited, and animated. All three writers (the frozen model, the live model, and the shopper) mutate the same contract, which is what makes shopper control a feature of the store itself rather than a settings page.

2.2 Two personalization models

First paint. A model trained on the H&M Personalized Fashion Recommendations dataset [8] reads a shopper's purchase history and assigns a deterministic persona (*Price Hunter*, *Trend Seeker*, *Mission Shopper*, or *Brand Loyalist*) from interpretable features (markdown affinity, novelty affinity, purchase frequency, category concentration, rebuy share). The persona selects the initial layout document, so the store arrives already merchandised: a Price Hunter sees sale-forward navigation and price-ascending sort; a Brand Loyalist sees a rail built around previously purchased colors and categories.

In-session. Two components adapt the store while the shopper browses. A LightGBM session model trained on the OTTO dataset [9] runs in the browser and re-derives per-field browsing decisions (grid density, comparison UI, quick-add, product-page density, cart reminders) after every click; each field activates only once the model's held-out accuracy for it crosses a threshold. Alongside it, a provisional persona is computed live from the shopper's own event stream (sale adds, price sorts, newest sorts, filters, searches, repeat views), nudged by the OTTO model's predicted browsing style. The persona is named only once enough evidence exists (five events minimum, a minimum score, then low/medium/high confidence rising with evidence), and only at medium confidence may it touch the layout, reordering navigation, hero, rails, and sort. Once the refresh loop has stored a persona scored on the shopper's whole history, that one takes over, so the panel, the terminal, the layout, and the concierge always name the same persona. A server-side refresh loop recomputes the shopper's document every four minutes, and every applied decision carries a human-readable reason string the UI surfaces on demand ("why am I seeing this?").

2.3 The concierge agent

The chat bubble streams responses from DeepSeek-V4.1-Flash via the Nebius API (with a Gemini fallback) through an NDJSON endpoint. Each request carries a grounded context payload (persona, decision reasons, bag contents, and what the shopper has told the concierge before), so questions like "why am I seeing this?" are answered from data.

The agent looks things up and acts through four server-side tools: `search_products`, `check_bag`, `add_to_bag`, and `checkout`. Products reach the model only as tool results from the full catalog, and an add is checked (product, color, size) before the model writes its reply; a refused add comes back with the reason and real alternatives. Because the bag lives in the browser, validated calls are handed to it as a typed action block, which the reply handler executes under the guards below. A provider without tool calling writes that block itself from a catalog in the prompt. Layout intent in the shopper's message (department, price direction, sale negation, color focus) is mapped onto contract edits by a deterministic parser and plays through the same staged animation as model-driven changes. The sentence is stored with the event and re-applied on later visits until the shopper asks the concierge to forget it.

2.4 The honesty layer

Deployed shopping agents fail in a specific way we observed repeatedly in development: the model claims an action it never performed. Our defense is layered, and each layer is independently testable:

(1) **Prompt.** Claims must be backed by a tool call; the prompt states that prose alone does nothing.
(2) **Parser.** The action block is extracted tolerantly across marker variants, fenced JSON, trailing prose, and partial streaming markers.
(3) **Validation.** Product ids, colors, and sizes resolve against the real catalog, on the server inside the tool call and again in the browser; invented references are refused.
(4) **Prose recovery.** If the model claims "I've added the Wool Trouser in black, size M" but emits no action, the named product and specs are extracted from the claim itself and the real state change executes.
(5) **Audit.** After every reply, claims in the text are compared against actual state transitions; a claim that did not execute is either recovered or an honest correction is appended.

Two guardrails sit underneath. Payment is gated on the shopper's *own words* (a typed confirmation or a button press, never a phrase inside a product request), and checkout capture is phase-guarded so that agent timers, manual payment, and toggles cannot race into a double charge. In our development traces this turned several real failure modes (phantom cart items, a double-captured order, misread "nope") into test cases; the system is covered by over 150 automated tests, including scripted multi-turn shopper sessions and the tool loop with the model mocked.

As a check, we run seven scripted transcripts through the production reply handler. Five reconstruct failure modes we observed during development (a claimed add with no emitted action, a hallucinated product, a premature checkout, a browse ask answered as a buy, a ghosted payment) and two are controls. A naive baseline that executes the emitted actions and shows the model's prose unchanged tells the shopper something false in 4 of the 7 and starts a payment the shopper did not ask for in 1. The audited pipeline does neither: one claim is recovered into a real state change and four are followed by an appended correction (Table 1). The transcripts are hand-written, so this shows the layer handles the failures it was built for, not how often a given model produces them.

We then ran the same comparison on the live model. Two disjoint sets of 240 generated requests (named and described purchases, browsing, invented products, unavailable colors or sizes, checkout with and without a bag, off-topic questions) were each sent once under both protocols: the model writing its own action block, and

Case	Naive	Audited
Claimed add, no action	false claim	recovered
Hallucinated product	false claim	corrected
Premature checkout	unasked payment	corrected
Browse answered as buy	false claim	corrected
Ghosted payment	false claim	corrected
Clean action (control)	true	true
Answer only (control)	true	true

Table 1: Seven scripted transcripts, run naively and through the audited pipeline. Naive: 4 false claims and 1 unasked payment. Audited: none of either.

the model using tools. Every reply was executed against real store state naively and through the audited pipeline, and what the reply tells the shopper was labelled by a separate model call. The first set exposed two gaps in the audit: a payment phrasing its claim pattern missed, and a two-part confirmation ("that's all, pay now") it did not recognize, which made the audit itself withhold a payment the reply had announced in 4 cases. We fixed both and report the second set, collected afterwards. Writing its own action block, the model told the shopper something false in 11 of 240 replies, every one announcing payment for an empty bag; the audit corrected all 11. With tools it made no false claim in 240, because the tool reports the empty bag before the reply is written. The first set gave 10 and 0. No reply in either set started a payment or an add the shopper had not asked for. Tools cost time: the median reply took 2.8 s without them and 6.1 s with them (2.6 s and 4.1 s on the first set). These are single samples from one model, labelled by a model; we read 36 of the labels and found 2 wrong, neither on a reply that counted as a false claim.

2.5 Observability and persistence

The simulator shell renders the store inside a fake-browser frame beside a persona panel, and narrates every event on a live terminal: tracker events, layout stages, LLM latency, agent actions, and payments. The same event stream (model inputs, emitted layouts, store events, live shopper layouts) persists to Snowflake, which serves as the substrate for the refresh loop and for post-hoc analysis of agent behavior. A `replay` command in the terminal reads a shopper's events and every stored layout version back as one timeline, showing which decisions each refresh changed.

The refresh loop itself runs inside Snowflake: a scheduled task invokes a Snowpark stored procedure every four minutes, rescores shoppers with new events using the frozen model bundle (staged as artifacts), and rewrites the layout, trending, and run-log tables. A second nightly job retrains a store-native next-department predictor on accumulated events, versions each trained model into a stage, and writes per-shopper predictions back to the warehouse, where the store reads them to order the shopper's For You products. The predictor is trained on little data so far and its quality is unmeasured; what we demonstrate is the loop from events to retraining to the storefront.

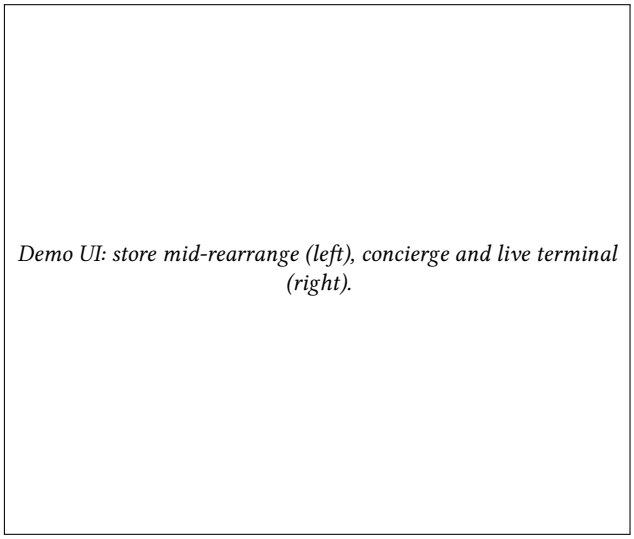


Figure 2: The demo interface. The layout diff animates in stages with tagged regions; the terminal logs every model, layout, agent, and payment event.

3 DEMONSTRATION

The demo runs the full system on a laptop: the SoCal Atelier store in a fake-browser frame, the persona panel, and the live terminal. Attendees can interact in three modes.

Watch the store learn. Selecting a shopper profile repaints the store pre-merchandised for that persona. Simulated browsing (viewing sale items, sorting by price, filtering men's) moves the provisional persona in the panel; within one refresh cycle the layout visibly rearranges, staged step by step, with each decision's reason string inspectable.

Talk back to the store. Attendees type layout intent into the concierge ("women's section first," "nothing too expensive," "show me more navy") and watch the nav slide, the hero swap, and the rails reorder. They can interrogate the result ("why am I seeing this shirt?") and get the actual decision reason.

Shop end to end. Attendees ask the concierge to shop ("buy the wool trouser in black, size M"), watch the item land in the bag with a checkout confirmation, answer "yes" or "nope" in their own words, and complete a mock Visa payment, including an auto-pay toggle they can flip mid-session. The terminal narrates every step, and attempted edge cases (invented product names, contradictory requests, toggling auto-pay mid-countdown) demonstrate the honesty layer live.

A 2-minute screencast of the full flow is available at TODO_VIDEO, and the source at https://github.com/AnselZeng/silicon-shoppers.

4 ETHICAL USE OF DATA

The demo uses public research datasets (H&M Personalized Fashion Recommendations [8]; OTTO multi-objective recommender [9]) and a synthetic product catalog; no real customer PII is processed.

Payments run through a mock Visa flow with no real financial instrument. The system collects no data from demo attendees. No human-subjects research was conducted.

5 CONCLUSION

Silicon Shoppers treats store layout as an agent-operable object: a typed contract that models and shoppers can both write, with every claim verified against state. We believe this direction, interfaces as action spaces and agents that must account for what they actually did, is where conversational commerce needs to go, and we welcome hands-on scrutiny.

REFERENCES

[1] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*, 2024.

[2] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024.

[3] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS Datasets and Benchmarks*, 2024.

[4] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: Measuring compositional web browsing skills of web agents. In *ICML*, 2024.

[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[6] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *NeurIPS*, 2024.

[7] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR*, 2024.

[8] H&M Group. H&M personalized fashion recommendations dataset. Kaggle, 2022. https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations

[9] OTTO GmbH. OTTO multi-objective recommender system dataset. Kaggle, 2022. https://www.kaggle.com/competitions/otto-recommender-system